Introduction — Chapter 1

2025 年 9 月

郭迎

guoying@bupt.edu.cn

School of Computer Science

北京郵電大學

Concepts（概念）: DB, DBMS, DBS, DBAS

- **Definitions**
  - **Database** (DB)
    - a collection of **interrelated** data
    - stored as files
  - **Database management system (DBMS),** consisting of
    - **DB**
    - **Programs** to access DB
- **DBMS** provides an environment for **store** and **retrieve** information
  - **definition** of structures for storage of information
  - **data manipulation** mechanisms
  - **data safety** mechanisms

- **Database system** (DBS)
  - Same definition as DBMS
  - DBS and DBMS are used **interchangeably**

- **DBS** manage collections of data:
  - **Highly valuable**
  - **Relatively large**
  - **Multiple users**, accessed by **at the same time**

Concepts: DB, DBMS, DBS, DBAS

- Definitions **in other textbooks**
  - **Database**(DB)
    - interrelated data*, stored as files*
  - **Database management system** (DBMS)
    - **system** to manage data in DB
    - **programs** to access the data in DB
  - **Database system** (DBS)
    - DB + DBMS + Users/Administers
  - **Database application system** (DBAS)
    - DB + DBMS + Application programs + Users/Administers

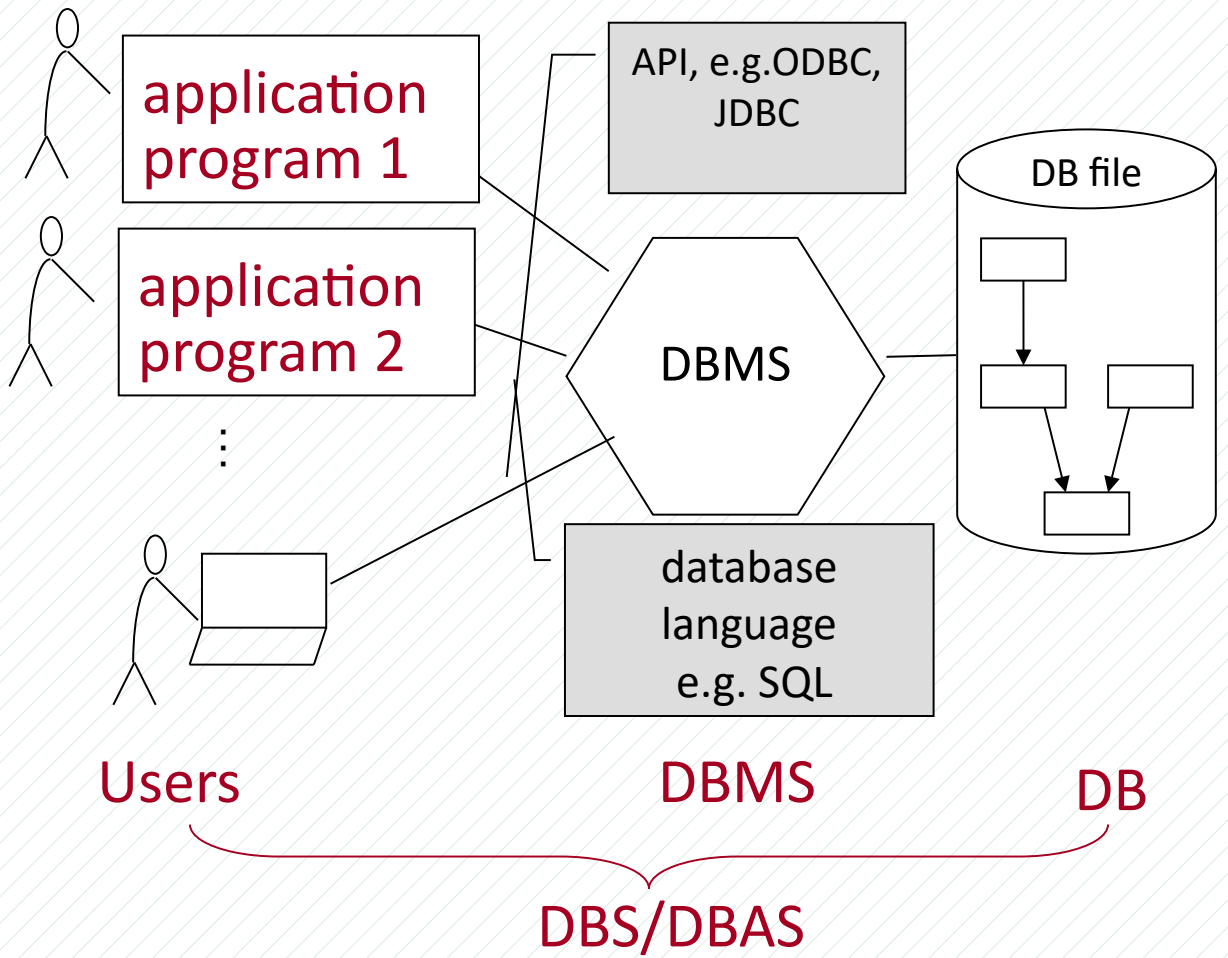


Fig.1.0.1 DBS and DBAS

1.1 Database Applications

- DBS can **manage** a **large collection** of data with various forms and structures
  - Databases touch all aspects of our lives
- **Enterprise Information**
  - Sales: customers, products, purchases
  - Accounting: payments, receipts, assets
  - Human Resources: Information about employees, salaries, payroll taxes.
- **Manufacturing**: management of production, inventory, orders, supply chain.
- **Banking and finance**
  - customer information, accounts, loans, and banking transactions.
  - Credit card transactions
  - Finance: sales and purchases of financial instruments (e.g., stocks and bonds; storing real-time market data

Database Applications Examples (Cont.)

- **Universities**: registration, grades
- **Airlines**: reservations, schedules
- **Telecommunication**
  - records of calls, texts, and data usage, generating monthly bills, maintaining balances on prepaid calling cards
- **Web-based services**
  - Online retailers: order tracking, customized recommendations
  - Online advertisements
- **Navigation systems**
- maintaining the locations of varies places of interest
- along with the exact routes of roads, train systems, buses, etc.

1.2 Purpose of Database Systems

DBAS were initially built directly on file systems, leading to:

- **Data redundancy and inconsistency**
  - data is stored in multiple file formats resulting in duplication of information
- **Difficulty in accessing data**
  - Need to write a new program to carry out new task, by using file access API
- **Data isolation**
  - Multiple files and formats
- **Integrity problems**
  - **Integrity** constraints (e.g., account balance > 0) become **buried** in program code rather than being stated explicitly
    - Integrity: semantic constraints
  - **Hard** to add new constraints or change existing ones

Purpose of Database Systems

- **Atomicity of updates**
  - Failures may leave database in **inconsistent** state with partial updates
  - Example: Transfer of funds from one account to another should **either** complete **or** not happen at all
- **Concurrent access by multiple users**
  - Concurrent access needed for **performance**
  - Uncontrolled concurrent accesses can lead to **inconsistencies**
- **Security problems**
  - Hard to provide user access to some, but not all, data

➢ **Database systems offer solutions to all the above problems**

Example: University Database

- **Consider university database** to illustrate all the concepts
- **Data consists of information about**:
  - Students
  - Instructors
  - Classes
- **Application program examples**
  - Add new students, instructors, and courses
  - Register students for courses, and generate class rosters
  - Assign grades to students, compute grade point averages (GPA) and generate transcripts

1.3 View of Data

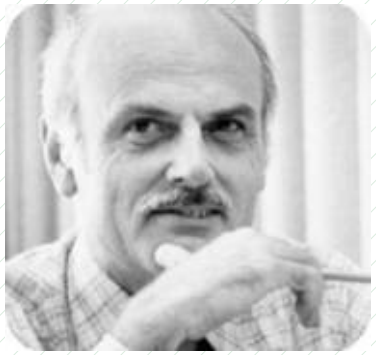
- **Provide** with **abstract** view of data.
  - **data models**
    - **conceptual tools** for **describing**:
      data, data relationships,
      data semantics, and consistency constraints
    - **specifications** for data **organization** and **access**
  - **data modeling**
    - **organize** application data, according to **data model**
    - by **data abstraction**
  - **data abstraction**
    - **A database modeling mechanism**
    - hide complexity of data structures to represent data through **view-logical-physical** levels of data abstraction

Data Model

- **Definition of data model**
  - 对数据组织与管理 / 使用**方式**的抽象描述，包括
    - 数据组织的**语法定义**，如数据项、数据项间的联系
    - 数据组织的**语义定义**，如完整性约束
    - **数据操作**
- **Relational** model
- **Entity-Relationship** data model (mainly for database design)
- **Object-based data** models (Object-oriented and Object-relational)
- **Semi-structured data** model (XML)

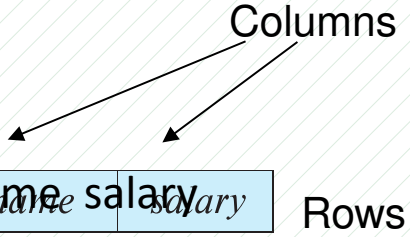
(1) Relational Model

- Classical and widely-used data model
- All the data is stored in various tables
  - proposed by Ted Codd



Ted Codd
Turing Award 1981

- Example of tabular data

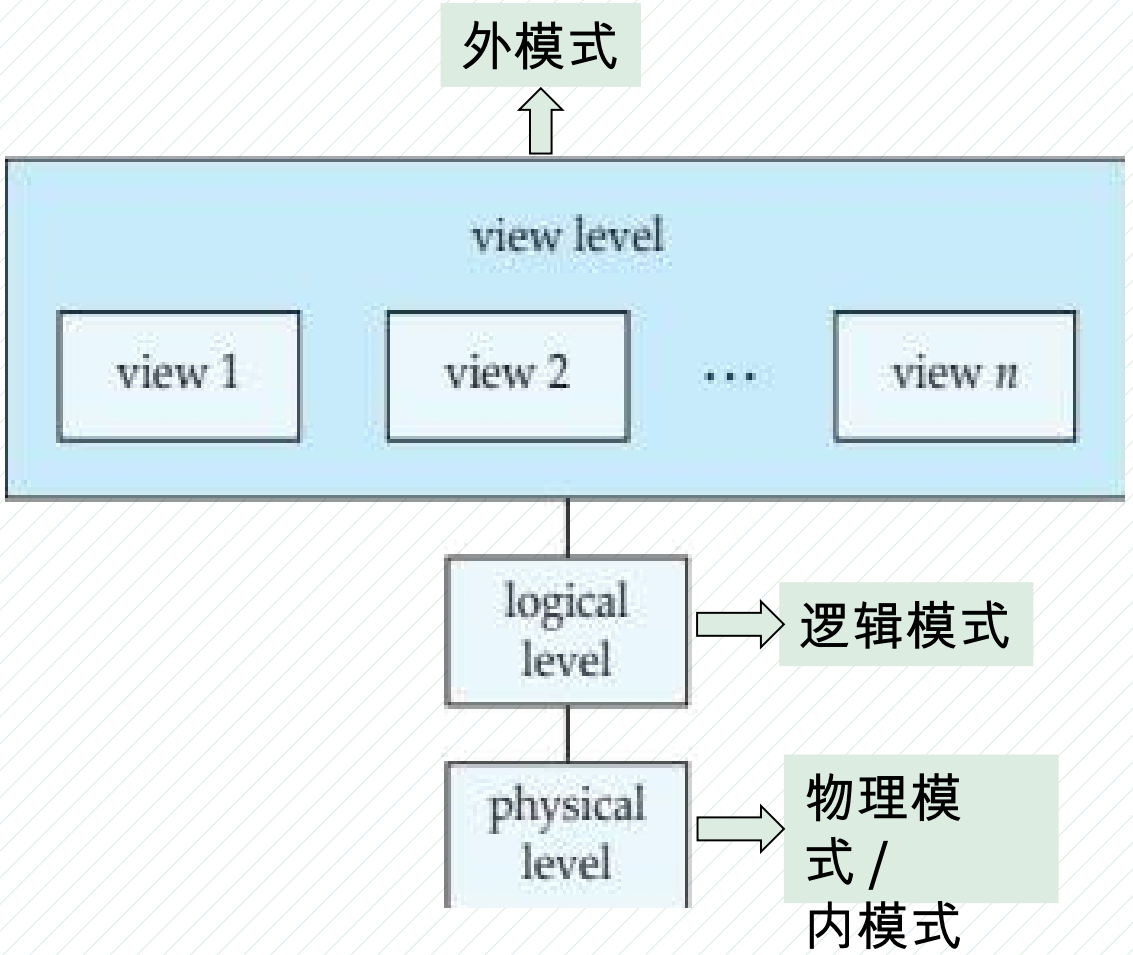


ID	name	dept_name	salary
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

(a) The *instructor* table

Levels of Abstraction

- **1. Physical level**
  - describes how a record is **stored**.
- **2. Logical level**
  - describes data stored in DB, and relationships among data.
- **3. View level**
  - hide details of data types
  - hide information for security purposes.



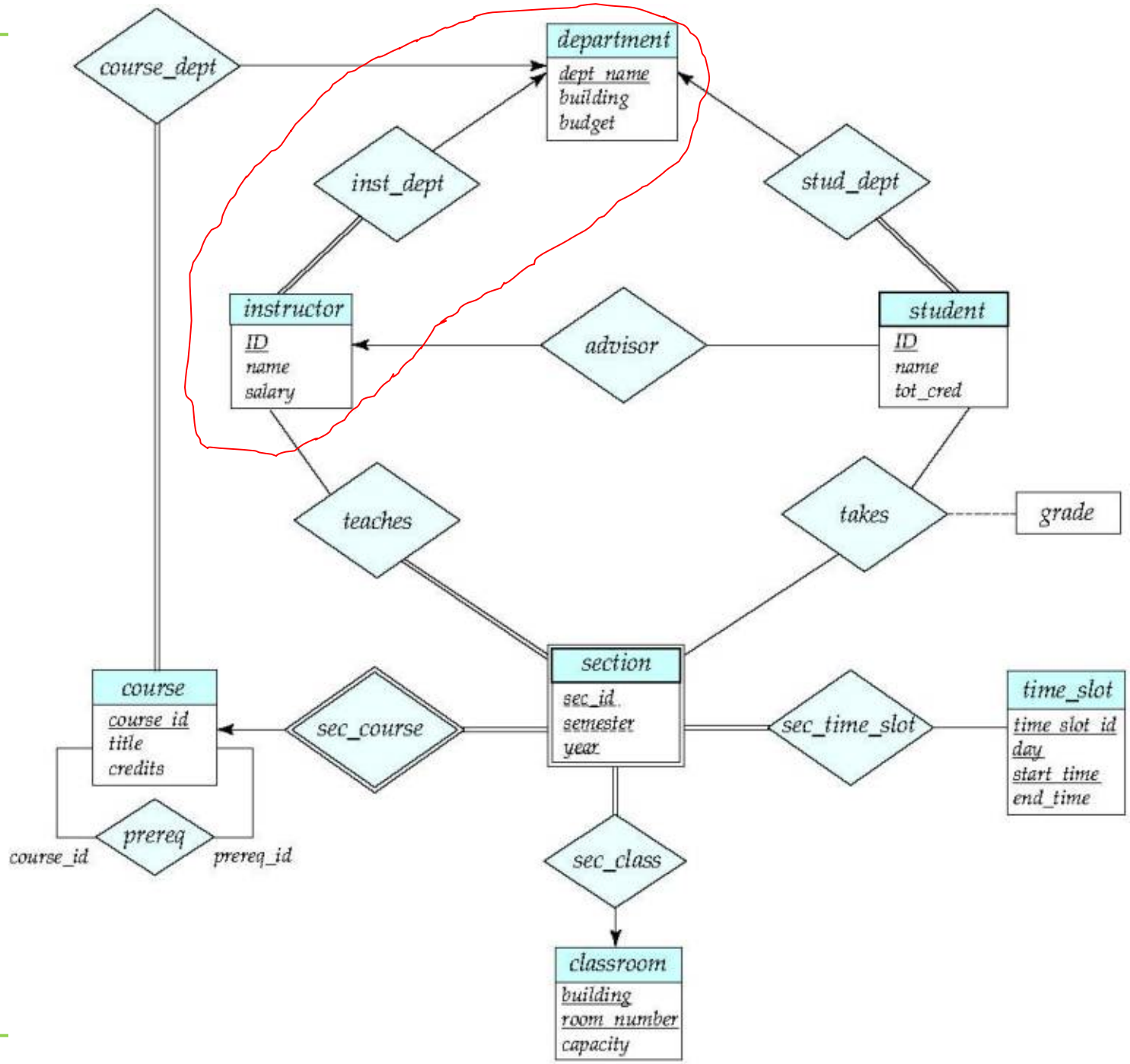
Levels of Abstraction

- **View level**

- **Example**: University database
  - **more than one** viewpoints, for a data object
    - $view_1$: *student*<id, name>
    - $view_2$: *student*<id, dept_name>
    - $view_3$: *student*<id, tot_cred>

(2) E-R diagram

Entity-Relationship (E-R)



Model, Schemas, Instances

- **Data modeling** produces **data** schemas(模式), **logical** and **physical** schema, which describe the **organization** of data in DB
- **Logical Schema** – the overall **logical** structure of the database
  - Example: DB consists of information about customers and accounts in ***bank*** and the relationship between them
- **Physical schema** – the overall **physical** structure of the database
  - how data is stored in database file, and how to access the database file
- **DB Instance** – the **actual content** of DB at a particular **point** in time
  - analogous to the value of a variable

➢ **Model** vs **schemas** vs **instance**

  - similar to types, variables and values of variables

Model, Schemas, Instances

- **Example**.
  - **relational data model**
    - $R=\{<a_1, a_2, ..., a_n>\}$
  - **relational data schema**
    - instructor=<ID, name, dept_name, salary>
  - **instance of schema**
    - <2222, Einstein, Physics, 95000>

特征： Data Independence

- **1. Logical data independence**
  - when **logical schemas** change, application-oriented **external schema** or application programs do not change
  - e.g.

student =<*id, name, tot_cred*>

is extended into

student =<*id, name,* ***age****, tot_cred*>,

or changed into

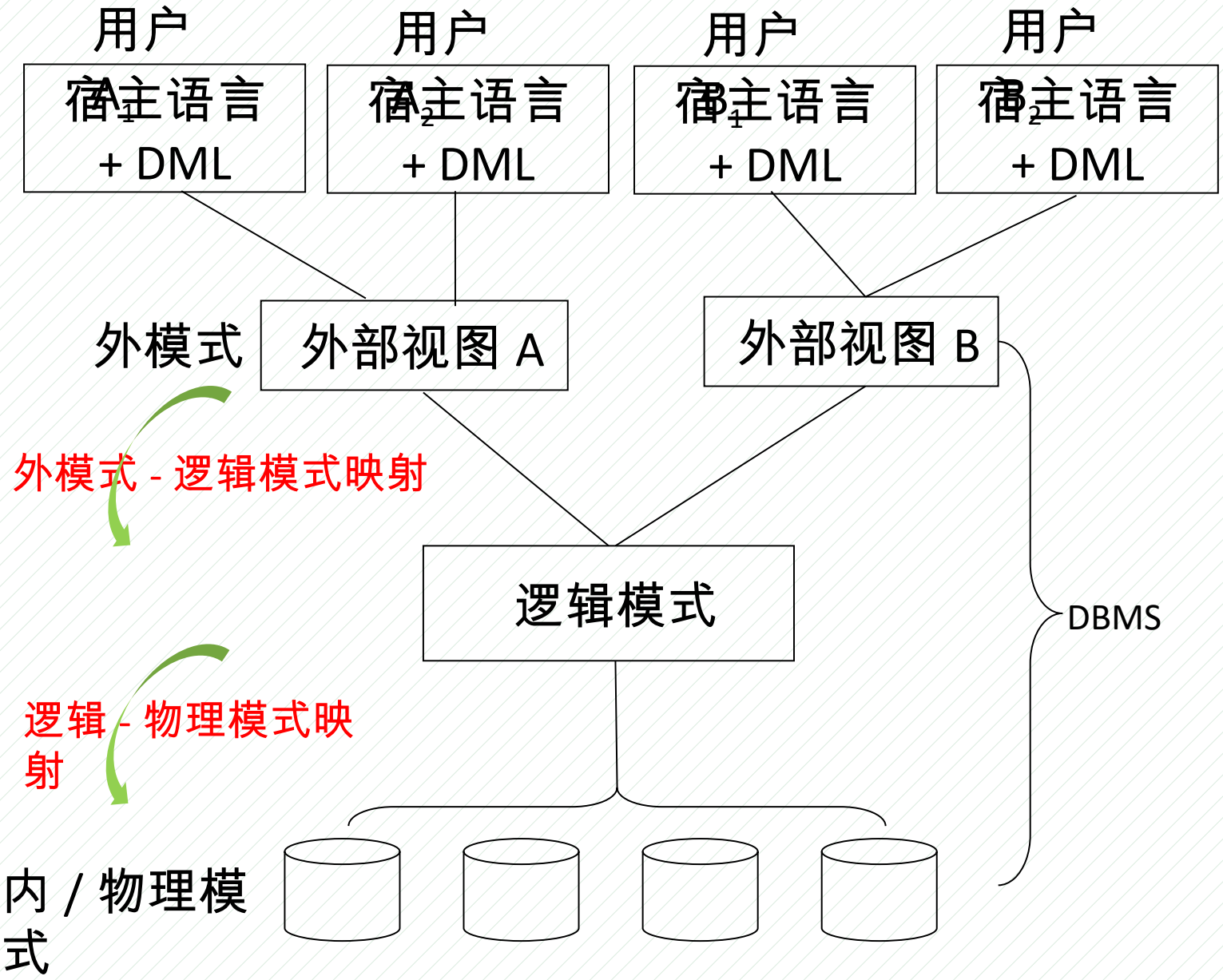
student =<***sid***, ***sname***, *age, tot_cred*>

- **2. Physical data independence**
  - application programs do not depend on physical schemas
  - when physical schemas such as index, storage location, and block-size change,
    application programs remain unchanged

- **目的应用： Database Migration**（数据库迁移）
  - change/replace DBMS platform, as an Example. from Oracle to openGauss

Data Independence

- 1. 逻辑模式→内模式间的映射与 physical data independence
  - **物理存储**变化，**调整**逻辑模式→内模式间的映射，保证逻辑模式不变，应用程序不变
- 2. 外模式→逻辑模式间的映射与 logical data independence
  - **数据逻辑结构**变化，**调整**外模式→逻辑模式间映射，保证外模式不变，从而应用程序不变



Database Migration (数据库迁移)

- Database Migration
  - **change/replace** DBMS platform, from Oracle to openGauss,
  - **avoiding** changes of logical schemas and application programs and maintaining data independence
- Migration Tasks
  - 应用数据迁移（ DB 模式 ）
  - 数据处理程序迁移（函数存储过程 ）
  - 模型重构优化
- Why
  - 国家网络空间**信息安全**，信息基础设施、系统软件（操作系统、数据库）
  - **自主可控**，**去 IOE**
  - 降成本
- 案例
  - 民生银行运维管理类系统数据库迁移项目
  - 中软国际金融行业国产化数据库迁移解决方案
  - 阿里云数据库应用迁移解决方案

1.4 Database Language: Data Definition Language (DDL)

- **Defining: database schema**

  Example: **create table** *instructor* (
  ID **char**(5),
  name **varchar**(20)**,**
  dept_name **varchar**(20),
  salary **numeric**(8,2))

- DDL compiler generates table templates stored in a **data dictionary**
- **Defining: Data dictionary** contains **metadata** (i.e., data about data)
  - Database schema
  - Integrity constraints
    - Primary key (ID uniquely identifies instructors)
  - Authorization
    - Who can access what

Database Language: Data Manipulation Language (DML)

- **Defining: DML** Language for **accessing and updating** the data
  - DML also known as query language
- There are two types of DMLs
  - **procedural DML** -- **what** data are needed and **how** to get those data
    - e.g. relational algebra
  - **declarative DML** -- **what** data are needed without specifying **how** to get
    - e.g. SQL
- Declarative DMLs are **easier** to learn and use than are procedural DMLs.
- Declarative DMLs are referred to as **non-procedural** DMLs

Database Language: SQL Query Language

- **Defining:** SQL query language is nonprocedural.
- A query takes as input **several tables** (possibly only one) and always returns **a single table**.
- Example to find all instructors in Comp. Sci. dept

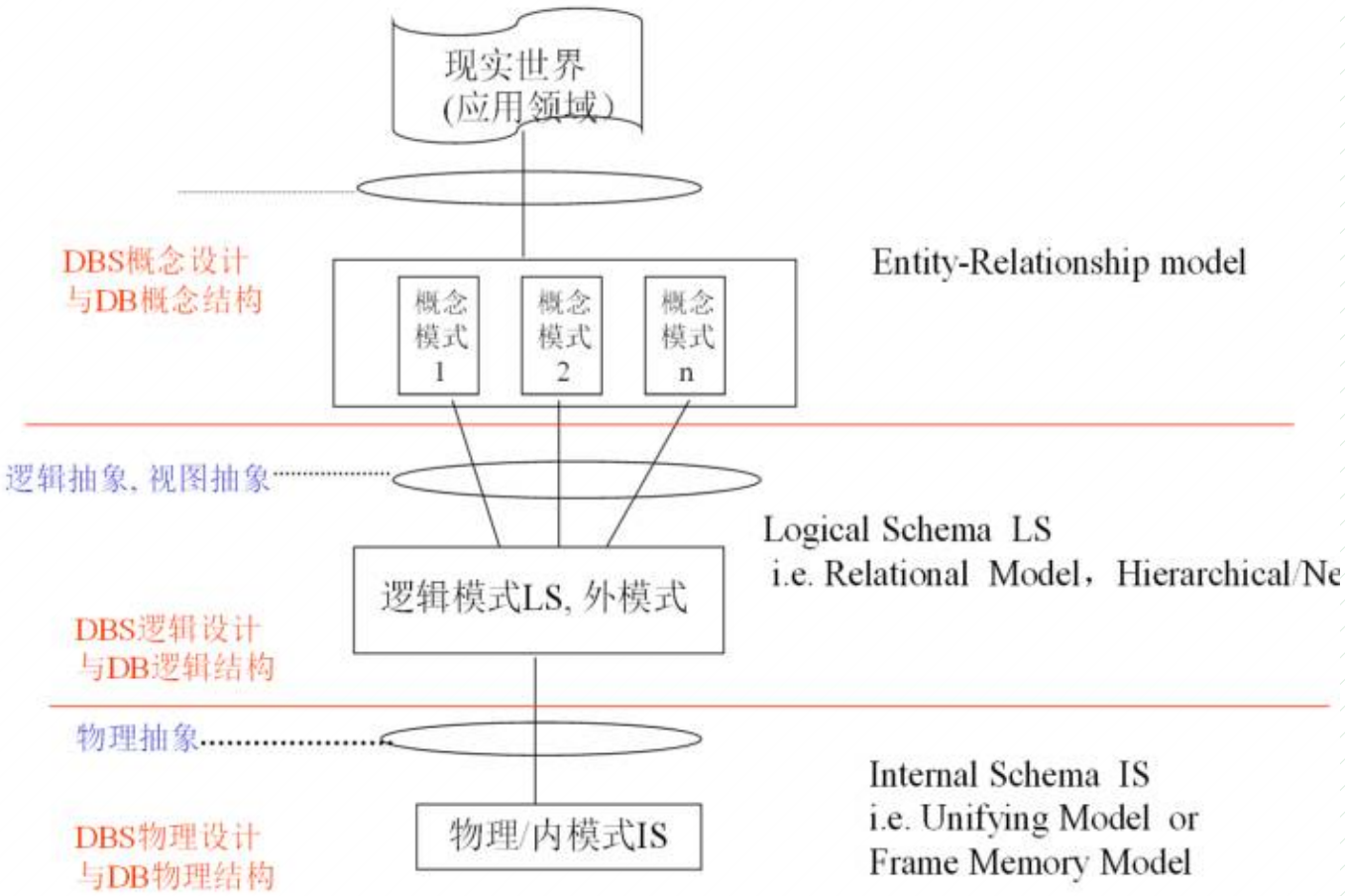
```
select name
from instructor
where dept_name = 'Comp. Sci.'
```

Database Language: Database Access from Application Program

- SQL is usually embedded in higher-level language
  - Application programs access databases through one of
    - Language extensions to allow **embedded SQL**
    - **Application program interface** (e.g., ODBC/JDBC) which allow SQL queries to be sent to a database
- SQL does not support actions such as input from users, output to displays, or communication over network.
  - Such computations and actions must be written in a **host language**, such as C/C++, Java or Python, with embedded SQL queries that access the data.
- **Application programs** -- are programs that are used to interact with DB.

1.5 Database Design

- 从保持 data independence 角度出发，根据 **data models** 定义数据规范形式，在 **view** 、 **logical** 、 **physical** 三层次，
- 通过 **data abstraction** ，
- 通过**概念设计，逻辑设计，物理设计**，
- 构造 DBS 的 E-R diagrams, external schema/logical schema , internal schema 的集合，
- 得到 conceptual DB ， logical DB ， physical DB



DBS设计与设计阶段：数据模型—数据抽象—数据模式

Database Design

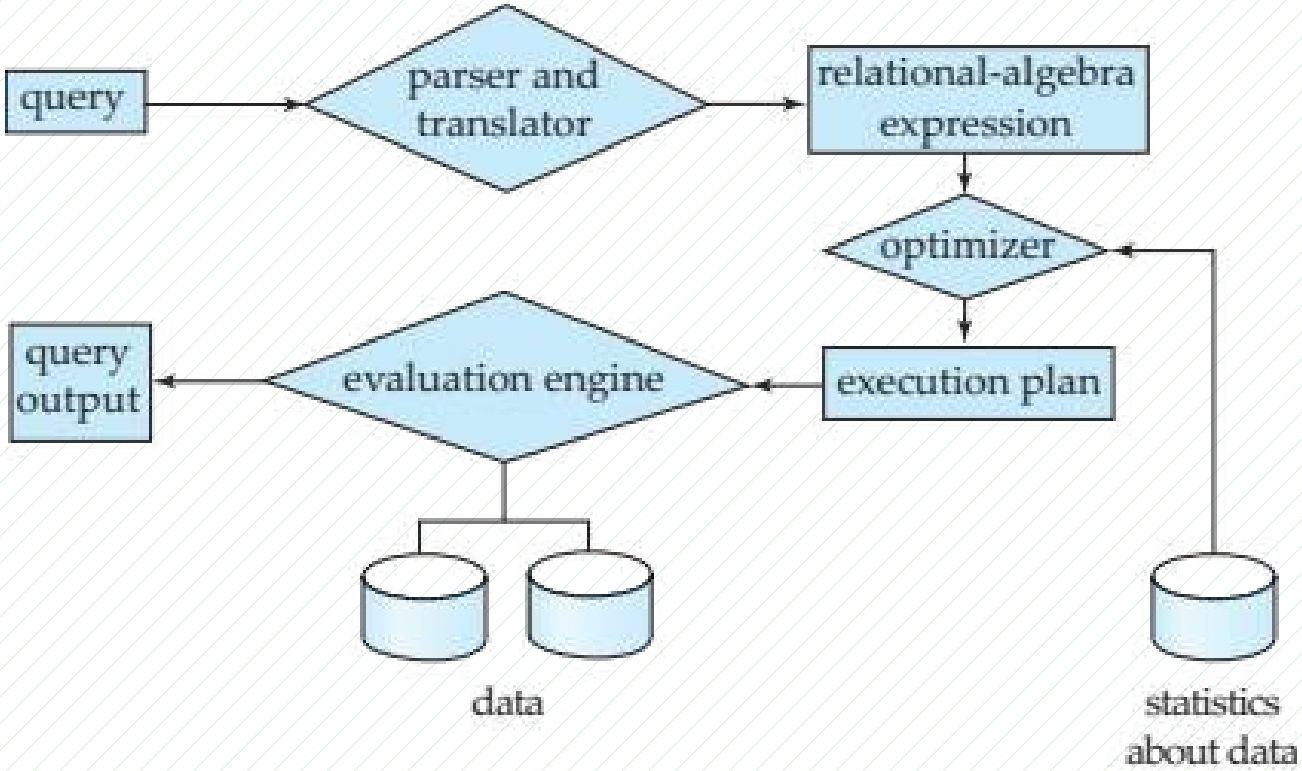
- **Logical Design**
  - Deciding on database schema. Resulting in a good relation schema.
    - business decision – What attributes should record in DB?
    - computer Science decision – What relation schemas should have and how should attributes be distributed among various relation schemas?
- **Physical Design**
  - deciding on physical layout of the database
  - depend on DBMS platform

1.6 Database Engine/DBMS

- **Defining:** A DBS is partitioned into modules that deal with each responsibility.
- The functional components of DBS can be divided into
  - 1) query processor
  - 2) storage manager
  - 3) transaction management

1 ） Query Processor

- The query processor include:
  - **DDL interpreter** -- interprets DDL statements and records the definitions in data dictionary.
  - **DML compiler** -- translates DML statements into an evaluation plan consisting of low-level instructions that query evaluation engine understands.
    - performs query optimization;
    - picks the lowest cost evaluation plan from among the various alternatives.
    - executes low-level instructions.



2 ） Storage Manager

- **Program module** that provides the interface between low-level data stored in DB and application programs, and queries submitted to the system.
- **Storage manager** is responsible to tasks:
  - Interaction with the OS file manager
  - Efficient storing, retrieving and updating of data
- **Storage manager include**:
  - authorization and integrity manager
  - transaction manager
  - file manager
  - buffer manager
- **Storage manager** implements as part of the physical system:
  - Data files -- store the database itself
  - Data dictionary -- stores metadata about the structure of the database.
  - Indices -- can provide fast access to data items.

3 ） Transaction Management

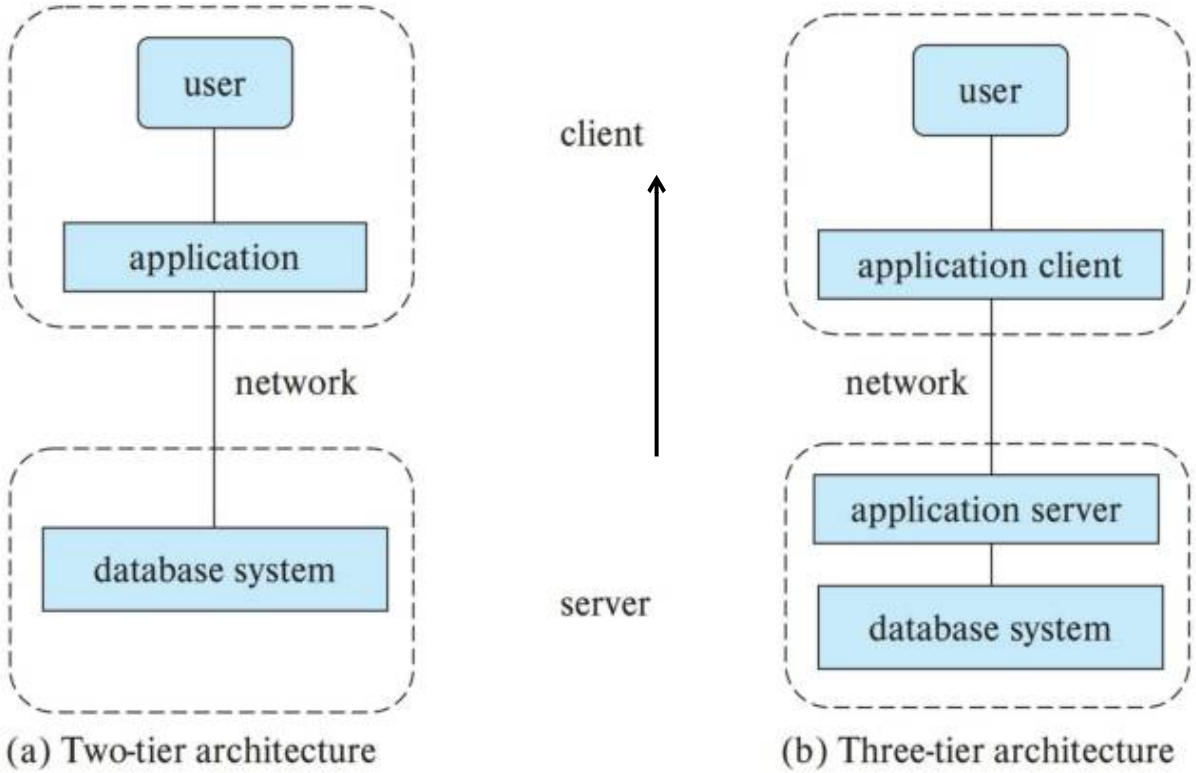
- **Transaction** is a collection of operations that performs a single logical function in DB application
- **Transaction-management** ensures that the database remains in a consistent (correct) state despite system failures (e.g., power failures and OS crashes) and transaction failures
- **Concurrency-control manager** controls the interaction among the concurrent transactions, to ensure the consistency of the database.

1.7 Database Architecture

- Centralized databases
  - One to a few cores, shared memory
  - e.g. SQL Server, MySQL, PostgreSQL
- Client-server, in network environments
  - One server machine executes work on behalf of multiple client machines
- Parallel databases
  - Many core shared memory
  - Shared disk
  - Shared nothing
- Distributed databases
  - Geographical distribution
  - Schema/data heterogeneity
  - e.g. openGauss, OceanBase

Database Architecture

- Database applications are partitioned
  - **Two-tier** architecture -- the application resides at client machine, where it invokes DBS functionality at server machine
  - **Three-tier** architecture
    - The client machine acts as a front end and does not contain direct DB calls.
    - The client end communicates with an application server.
    - The application server communicates with a DB system to access data.

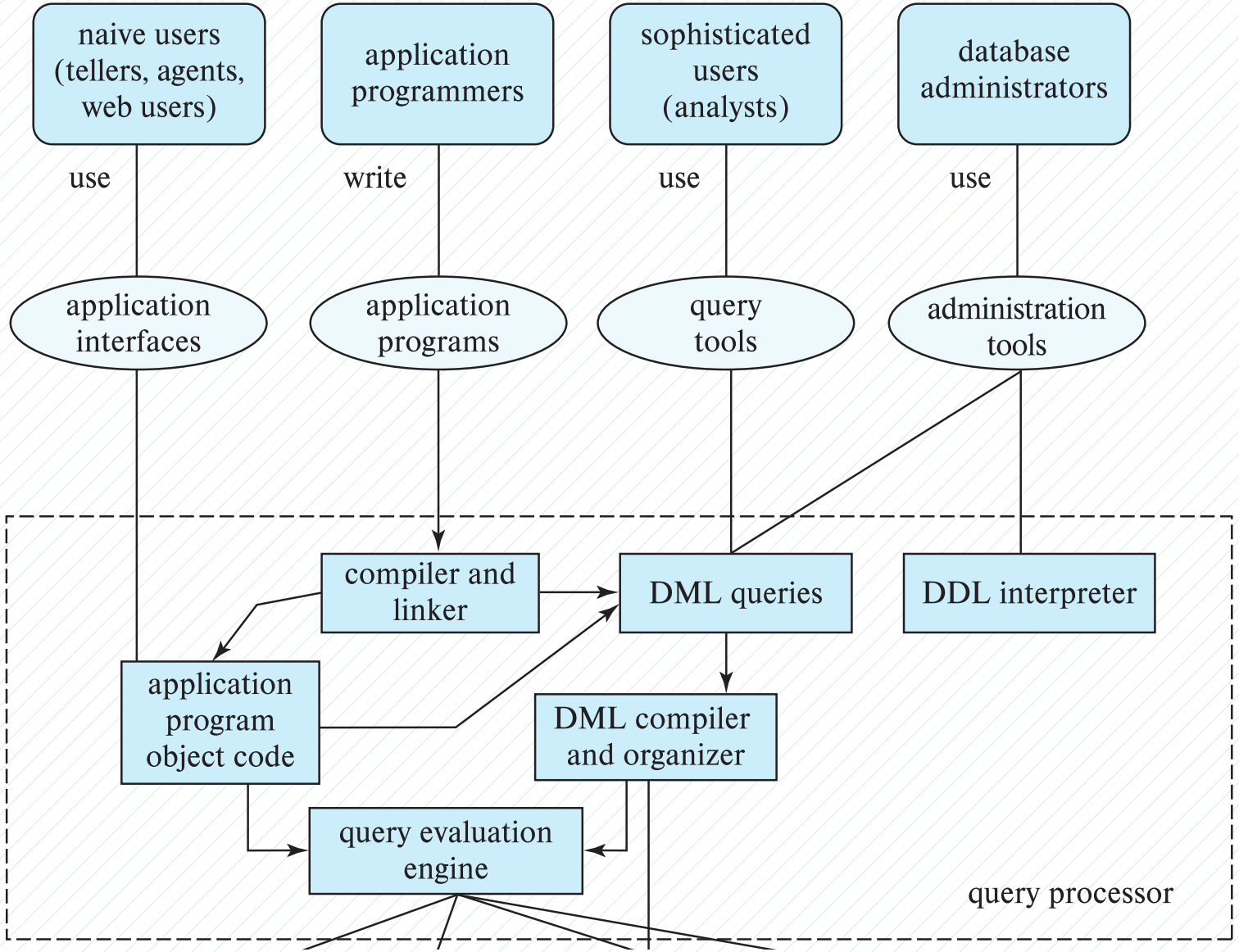


(a) Two-tier architecture (b) Three-tier architecture

1.8 Database Users and Administrators

- **Defining:**

  Four categories of users
  - 系统管理员 (sys admin)
  - 数据库管理员（ DBA ）
  - 应用程序用户
    （ application user ）
  - 数据库开发人员
    （ database developer ）



Database Administrator/DBA

- A person who has central control over the system is called a **database administrator (DBA).** Functions of a DBA include: **DBS 运维**
  - schema definition
  - storage structure and access-method definition
  - schema and physical-organization modification
  - granting of authorization for data access
  - routine maintenance
  - periodically backing up the database
  - ensuring that enough free disk space is available for normal operations
  - upgrading disk space as required
  - Monitoring jobs running on the database

1.9 History of Database Systems

- 1950s and early 1960s
  - Data processing using magnetic tapes for storage
    - Tapes provided only sequential access
  - Punched cards for input
- Late 1960s and 1970s
  - Hard disks allowed direct access to data
  - Network and hierarchical data models in widespread use
  - Ted Codd defines the relational data model, in 1971
    - win the ACM Turing Award for this work
    - IBM Research begins System R prototype
    - UC Berkeley (Michael Stonebraker) begins Ingres prototype
    - Oracle releases first commercial relational database
  - High-performance (for the era) transaction processing

Charles W. Bachman

Turing Award 1973

主持设计开发最早的网状数据库管理系统 IDS，DB 三级模式

Ted Codd

Turing Award 1981

关系数据库之父

James Gray

Turing Award 1995

数据库事务 / 完整性 / 安全性，SQL Server 7.0

对现代数据库系统底层的概念与实践所做出的基础性贡献；SQL Server/Sybase 奠基人

Michael Stonebraker

Turing Award 2014

History of Database Systems (Cont.)

- 1980s
  - Research relational prototypes evolve into commercial systems
    - SQL becomes industrial standard
  - Parallel and distributed database systems
    - Wisconsin, IBM, Teradata
  - Object-oriented database systems
- 1990s
  - Large decision support and data-mining 【数据挖掘】 applications
  - Large multi-terabyte data warehouses 【数据仓库】
  - Emergence of Web commerce
- 2000s
  - Big data storage systems
    - Google BigTable, Yahoo PNuts, Amazon,
    - "NoSQL" systems.
  - Big data analysis: beyond SQL
    - Mapreduce and friends
- 2010s
  - SQL reloaded
    - SQL front end to Map Reduce systems
    - Massively parallel database systems
    - Multi-core main-memory databases
- Recent years
  - 数据仓库 / 数据中台 / 数据集市 / 数据湖
  - **云原生数据库 , cloud-native**

Appendix 1-1 数据库发展历史

数据库萌芽时代

底层算力：大型机 <100 台；

用传统的表格、文件系统管理数据。问题：随着数据量的增长，表格和文件越来越难以满足多用户共享数据、快速检索的需求

1964 通用电气公司推出**网状**数据库系统 IDS (Integrated Data Storage) ，集成数据存储），可以被称之为世界上第一个数据库系统，成功将数据从应用程序中独立出来、集中管理

1968 IBM 推出第一个**层次**数据库系统 IMS (Information Management System, 信息管理系统），目的是帮助 NASA 管理阿波罗计划中 SatumV 月球火箭和 Apollo 航天器库存巨大的物料清单 (BOM) ，是最早的事务管理系统

传统商业数据库时代

底层算力：小型机 <1 万台；

网状数据库和层次数据库在数据独立性和抽象级别上有欠缺，并且对编程人员要求极高。将数据应用独立于硬件存储，数据库用户或应用程序无需知道数据结构就能查询数据的想法，催生了关系型数据库

1970 埃德加 · 科德（EdgarFCodd）发表论文“ A Relational Model of Data for LargeShared Data Banks ”，提出“关系数据模型”，用数学理论奠定了关系型数据库的基础被称为“关系型数据库之父”

1974 瑞 · 博伊斯（RayBoyce）和道 · 钱伯伦（DonChamberlin）在科德的基础上，提出了操作关系型数据库的标准语言 SQL (Structured Query Language ，结构化查询语言) 。目前几乎所有关系型数据库产品都支持 SOL

1977 拉里 · 埃里森（LamyEllison）创办公司 SDL ，三年后发布了第一个商用 SOL 关系型数据库 Oracle 。 1982 年公司更名为 Oracle ，并逐渐成为商业数据库的领导厂商

1983 IBM 发布 DB2 数据库。此后的 30 年， DB2 都是最主流的数据库之一，全球 500 家最大的企业中，八成的企业级应用使用 DB2 数据库

1989 微软推出 SOL Server ，具有方便可伸缩及相关软件集成度高等优点，从笔记本电脑到大型多处理器的多种算力硬件上都能使用，与 Oracle 、 DB2 并列占据了商业数据库的前三甲

Appendix 1-1 数据库发展历史

开源数据库时代

底层算力： PC 机、 X86 服务器 <1 亿台；

开源文化在欧美盛行，其更具灵活性、创新性、更快选代的优势改变了软件产业的运行方式。开源社区中开发者、贡献者和用户共同促成了数据库发展的飞轮效应

1986 加州大学伯克利分校启动 Postgres 项目，源于 1974 年 **Michael Stonebraker** 开发的 Ingres ； 8 年后两名香港研究生安德鲁 · 余 (Andrew Yu) 和乔立 · 陈 (Jolly Chen) 在 Postgres 中添加了 SQL 翻译程序， **PostgreSQL** 数据库系统诞生。它具有高兼容性且开源，直到 2021 年都是第四大最受欢迎的数据库

1990 蒙蒂 (Michael"MontyWidenius) 认为已商用数据库的速度难以令人满意，重写了一个 SQL 查询引擎，并在 1996 年发布了 **MySQL**1.0 。 2005 年， MySQL5.0 进入高性能数据库行列，逐渐成为最受欢迎的开源数据库和关系型数据库， 90% 以上的互联网公司用过 MySQL

1998 加州关系型数据库呈现出难以支撑海量数据和复杂类型数据的局限性，卡洛 · 斯齐 (Carlo Strozzi) 开发了一个轻量、开源、不提供 SOL 功能的数据库，被称为 NoSOL (Non-Relational ，非关系型数据库) 。 NoSOL 的兴起，诞生了开源数据库 MongoDB (文档型) 、 Redis (内存数据库) 等

分布式数据库时代

底层算力： PC 机、 x86 服务器 <10 亿台；

数据规模爆炸式增长，单机数据库难以满足用户对资源利用率、可扩展性、可靠性的要求，数据库开始普遍建立在网络之上，产生了分布式数据库 (distributed DB)

1979 美国计算机公司在 DEC 计算机上实现了世界上第一个分布式数据库系统 SDD-1

1987 C.J. 戴特 (C.JDate) 提出分布式数据库系统应遵循的原则 CAP ，被认为是分布式数据库系统的理想目标

2003 | 2006 分布式数据库迎来真正的突破。 Google 发表了三篇论文，分别关于 Google File System (分布式文件系统) 、 Google MapReduce (分布式计算框架) 和 Google BigTable (分布式数据存储系统) ，奠定了分布式数据系统的基础。 2007 年 HBase 的诞生，就是 Google Big Table 的开源实现

2007 亚马逊发表 Dynamo 论文，基于分布式系统的设计原则， DynamoDB 就是在此基础上推出的可随意扩展、高可靠的 NoSOL 数据系统

2012 NoSQL 数据库虽然解决了关系型数据库高并发、多结构化数据存储等问题，但牺牲了关系型数据库事务处理的一致性。 Google 发布 Spanner 和 F1 系统论文，创新性地在第一代 NoSOL 系统基础上引入了 SOL 和分布式事务，NewSOL 数据库诞生

Appendix 1-1 数据库发展历史

云原生数据库时代

底层算力：云 + 端 >100 亿台；

云计算的发展让数据库部署在“云上”成为可能，由于云原生数据库可以共享架构，极大堆据库的存储能力，具有低成本、免部署、高性能、更安全等特点

2014 国内首个关系型云原生数据库（阿里云）PolarDB 开始孵化，并于 2017 年发布， 2018 年正式商用，100TB 数据容量提供了 10 倍于传统商业数据库的性价比”

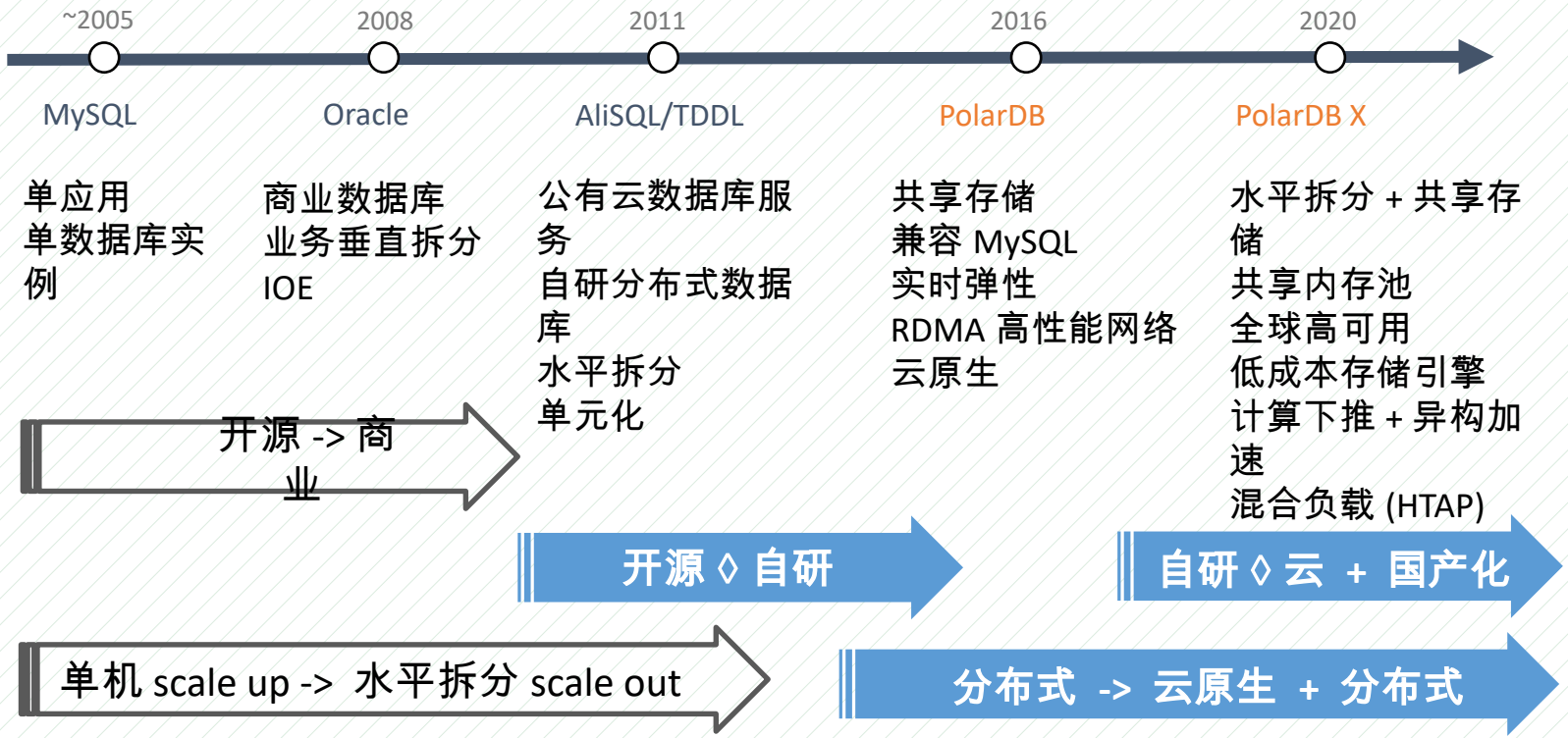
2015 亚马逊发布首个云原生关系型数据库 Aurora ，首先提出“日志即数据库”的理念，减少了网络消耗

2019 Gartner 发布“ The Future of the Database Management System(DBMS) Market Is Cloud ”报告，明确提出传统数据库已经过时。 2022 年全球 75% 的数据库会跑在云上

2020 全球企业数据库市场规模增长 17.1% ，达到 648 亿美金，其中增量的 90% 来自云原生数据库，云原生数据库市场规模超过传统数据库，成为数据库发展的分水岭

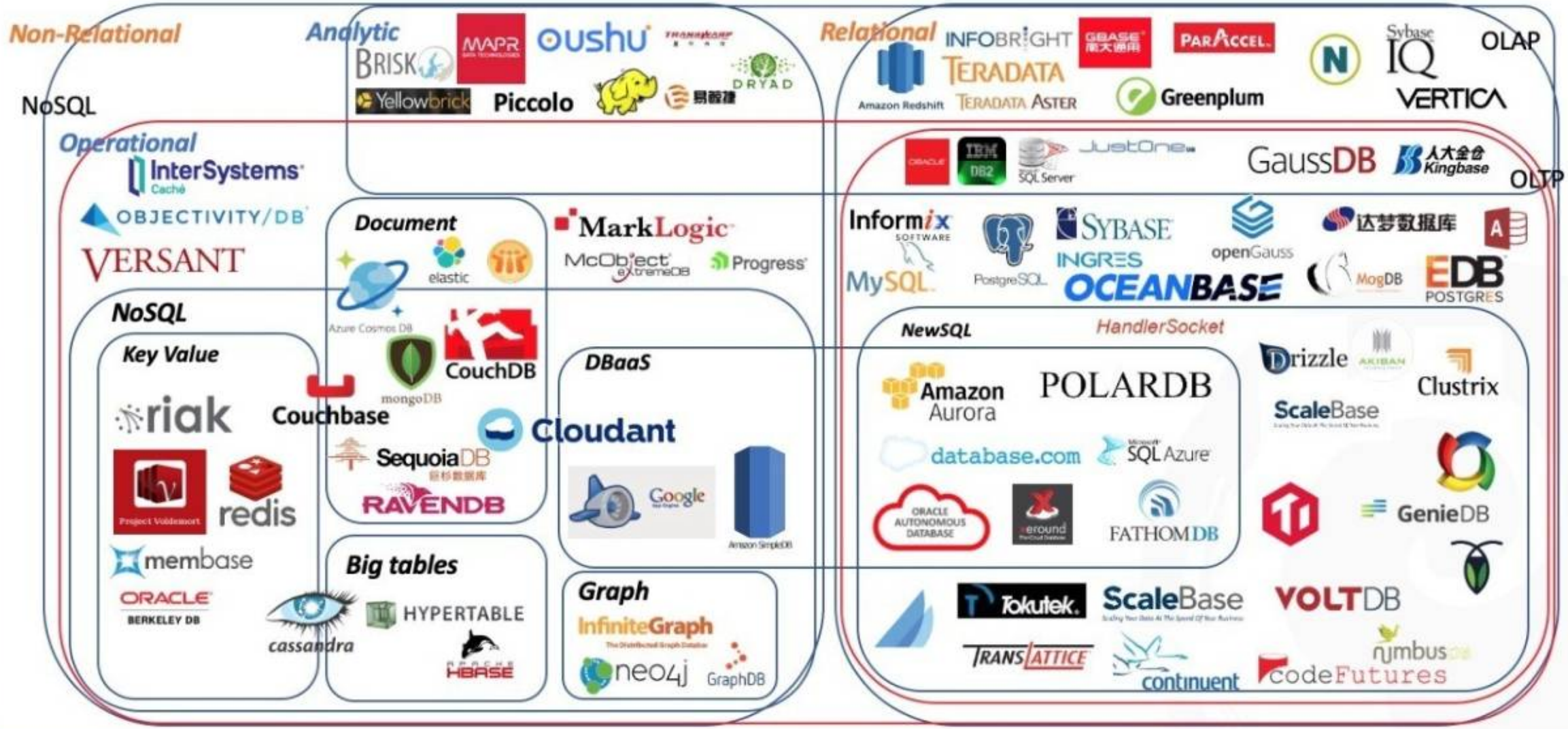
Appendix 1-1 数据库发展历史

阿里巴巴在线事务处理数据库演进路线

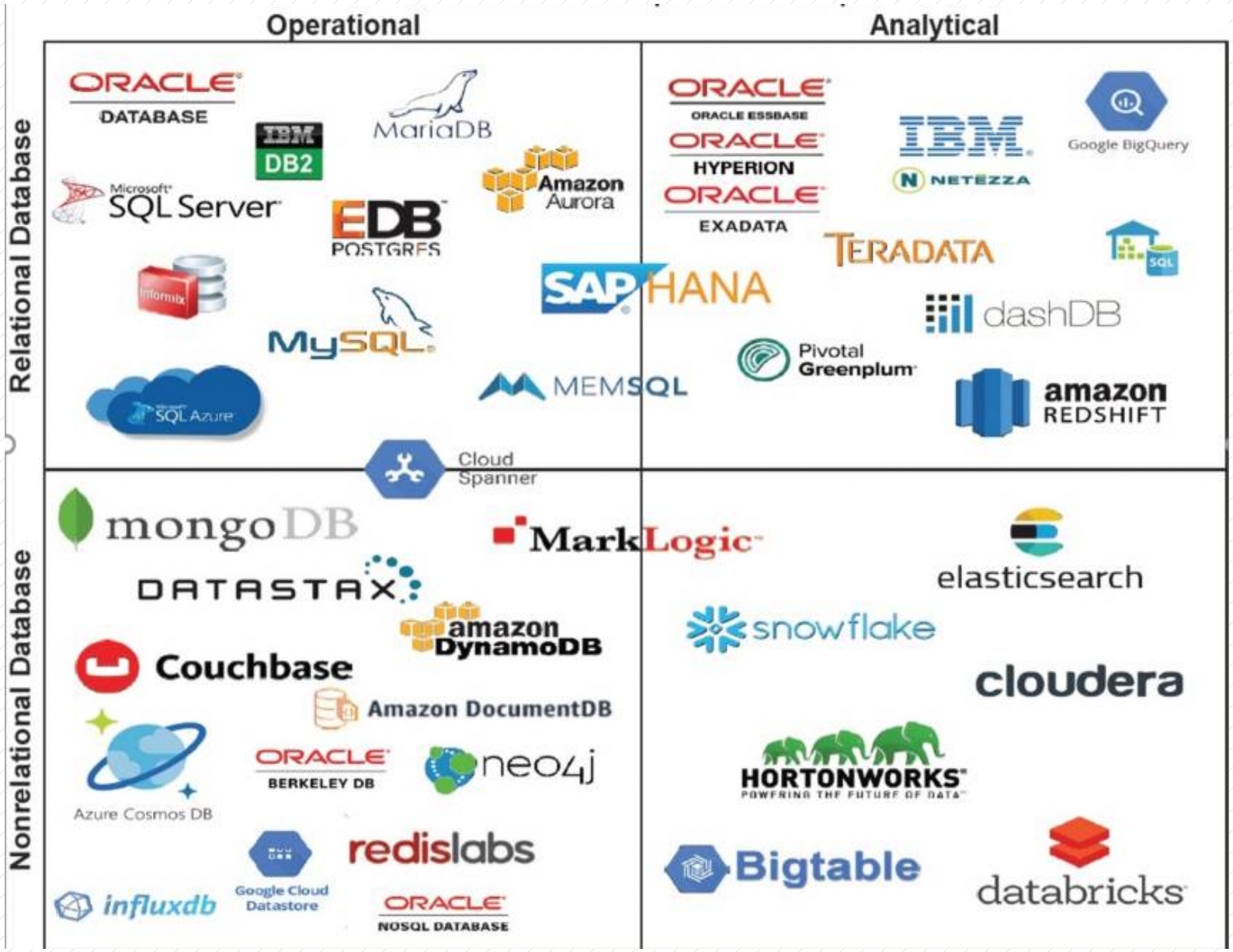


高并发：支撑双十一 8700 万 / 秒峰值查询　　**事务**：峰值 55w 笔交易

数据库的发展史

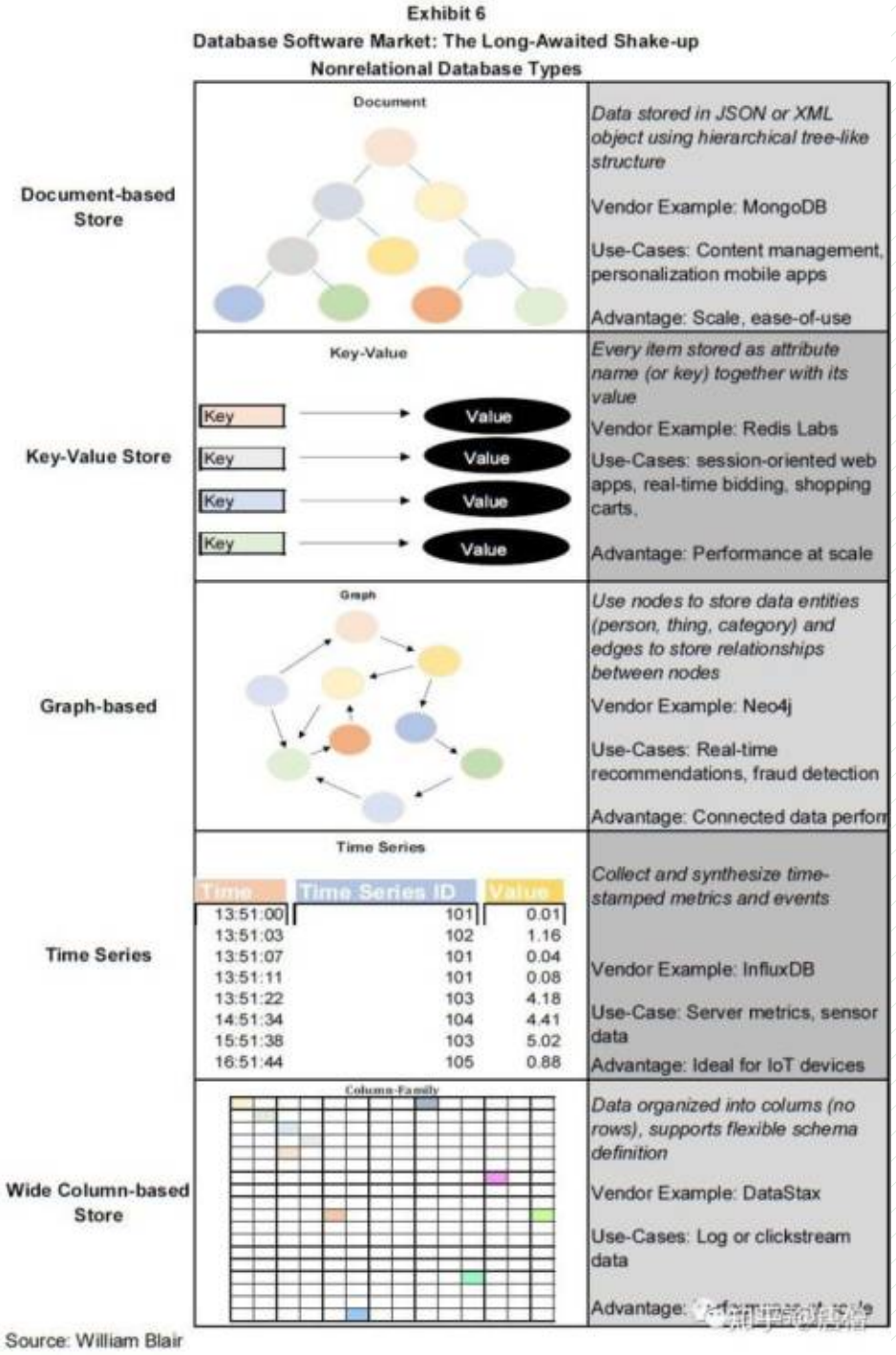

Non-Relational
NoSQL
Analytic
BRISK
MAPR
oushu
Yellowbrick
Piccolo
DRYAD
Relational
INFOBRIGHT
TERADATA
Amazon Redshift
TERADATA ASTER
PARACCEL
Greenplum
Sybase IQ
VERTICA
OLAP
Operational
InterSystems Caché
OBJECTIVITY/DB
VERSANT
Document
elastic
Azure Cosmos DB
MarkLogic
McObject eXtremeDB
Progress
NoSQL
Key Value
riak
Couchbase
Project Voldemort
redis
membase
ORACLE BERKELEY DB
cassandra
mongoDB
CouchDB
SequoiaDB
RAVENDB
Cloudant
DBaaS
Google
Amazon SimpleDB
Big tables
HYPERTABLE
APACHE HBASE
Graph
InfiniteGraph
neo4j
GraphDB
ORACLE
DB2
SQL Server
JustOne
GaussDB
Kingbase
OLTP
Informix SOFTWARE
MySQL
PostgreSQL
SYBASE
INGRES
OCEANBASE
openGauss
达梦数据库
MogDB
EDB POSTGRES
NewSQL
HandlerSocket
Amazon Aurora
POLARDB
database.com
SQL Azure
ORACLE AUTONOMOUS DATABASE
FATHOM DB
Drizzle
AKIBAN
Clustrix
ScaleBase
GenieDB
Tokutek
ScaleBase
VOLTDB
TRANSLATTICE
continuent
nimbus
codeFutures
数/据/驱/动 成/就/未/来
Confidential - Enmotech Internal/Restricted/Highly Restricted

1.10 Database Products&Markets



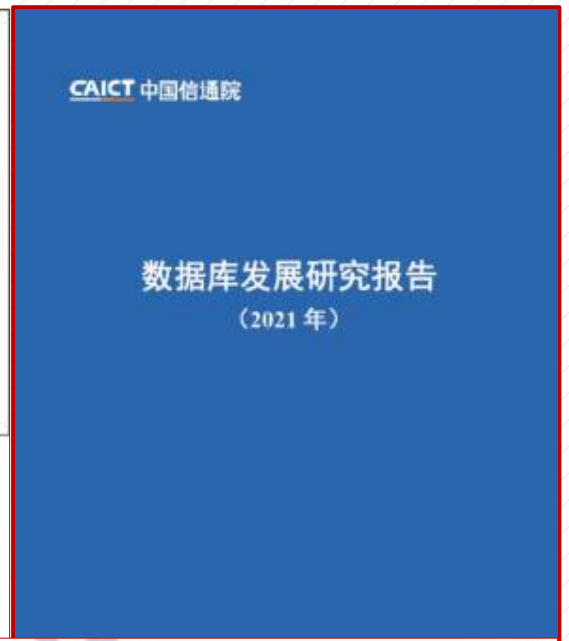
Database Products&Markets

- Relational databases dominates markets
- New types of non-relational (NonSQL) databases are emerging
  - document
  - key-value
  - graph
  - time series
  - main-memory
  - column-stored
  - multi-model



国产数据库：机遇与挑战

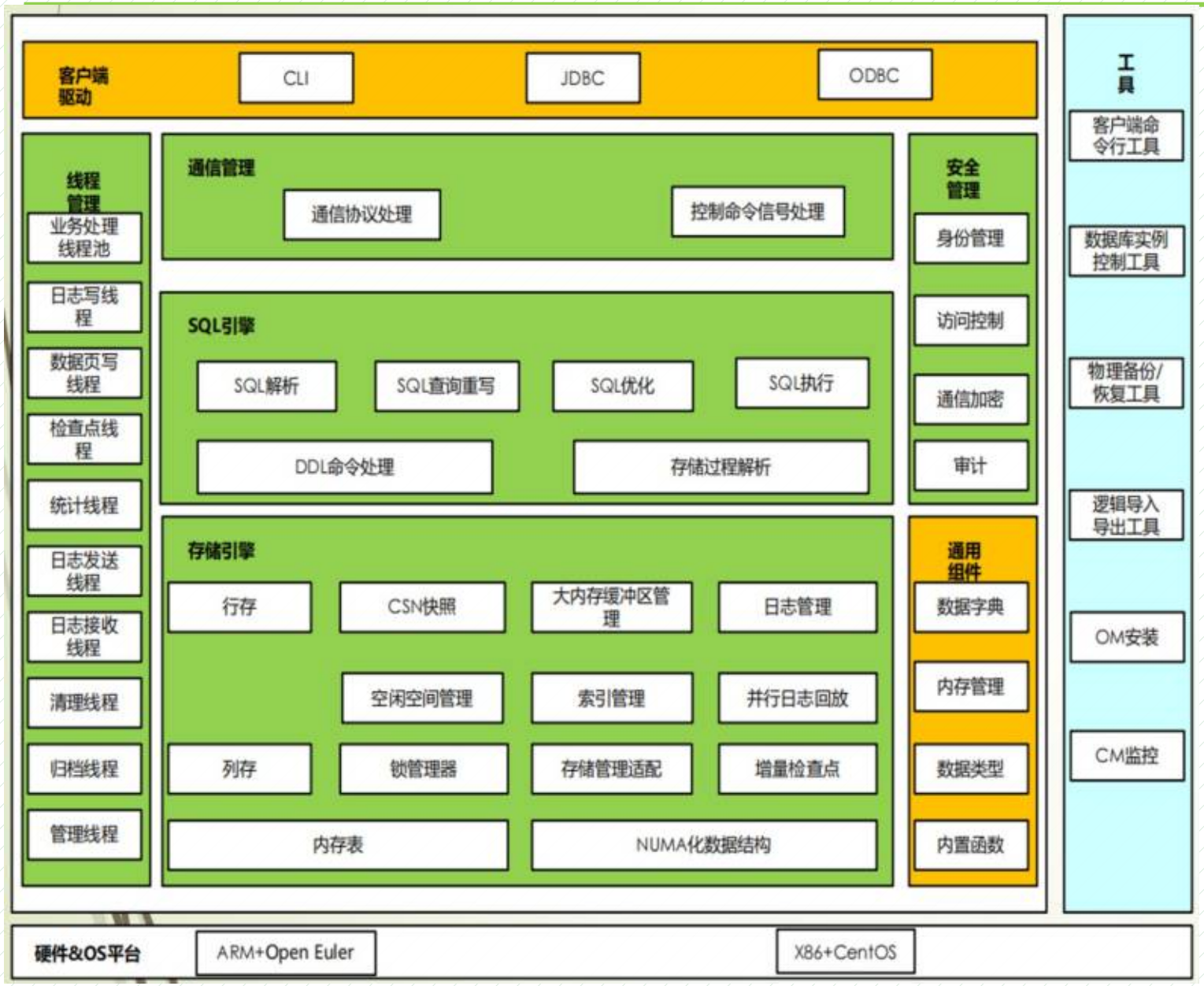
关系型数据库（Relationa DBMS）	OLTP	阿里云 POLARDB OceanBase 腾讯云 CynosDB 腾讯云 TDSQL	openGauss HUAWEI TaurusDB ZTE中兴 GoldenDB inspur浪潮 K-DB	达梦数据库 人大金仓 Kingbase GBASE GBase 8t 神舟通用	TRANSWARP KunDB SequoiaDB QINGCLOUD RadonDB	SINOREGAL 易鲸捷 HighGo DB UXSINO 优炫软件	热璞科技 HOTPU 极数云舟 ShinData 新数科技
	OLAP	阿里云 AnalyticDB 腾讯云 Tbase	openGauss GaussDB 200 TRANSWARP ArgoDB	GBASE GBase 8a	oushu HighGo DW	Kyligence 人大金仓 Kingbase	
非关系型数据库（NoSQL）	Graph DBMS	阿里云 GDB	TRANSWARP StellarDB	GeaBase			
	Key-Value	HUAWEI GeminiDB	腾讯云 TcaplusDB				
	Document						
	Wide column		阿里云 Tablestore				
	Time Series		阿里云 TSDB				



来源：中国信息通信研究院，2021 年 6 月

图 8 中国数据库市场规模及增速

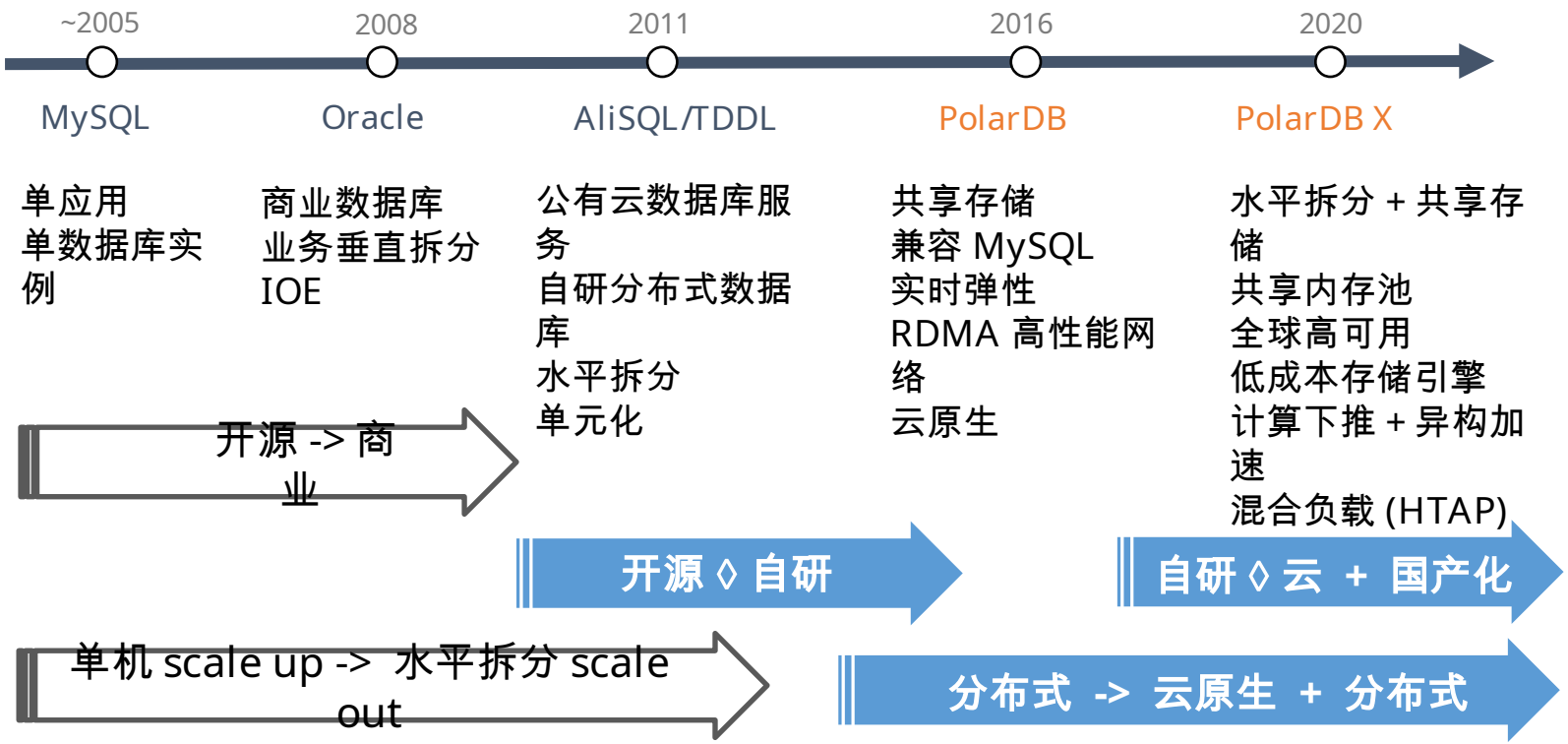
openGauss 架构：“脱胎换骨”深度改造



- openGauss/GaussDB 源自 PostgreSQL9.2 和 PG XC
- openGauss 总代码量 120w 行，其中内核代码 95w 万行代码
  - 内核代码中修改和新增 70w 行核心代码，保留了 PostgreSQL 的接口和公共函数 25w 行
- openGauss 着重在架构、事务、存储引擎、优化器、鲲鹏芯片优化上进行修改完善
- openGauss 不是 Postgre SQL 的简单增强版，通过“换骨”改造、“换血”优化，从根本上解决 PostgreSQL 原生架构带来的缺陷

Database Development

阿里巴巴在线事务处理数据库演进路线



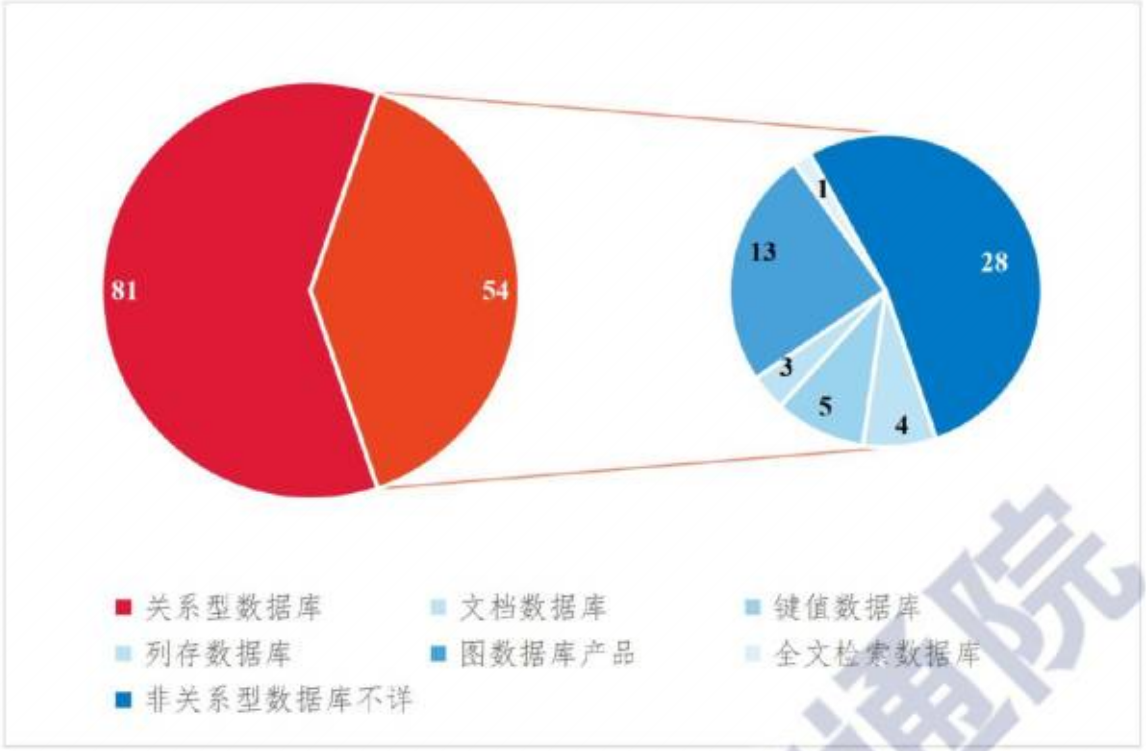
高并发：支撑双十一 8700 万 / 秒峰值查询　　**事务**：峰值 55w 笔交易

2024 年墨天轮国产数据库流行度排行榜

排行	上月	半年前	名称	模型	属性	三方评测	生态	专利	论文	得分	上月	半年前
1	1	1	TiDB +	关系型	HP			15	23	580.95	-16.79	+1.42
2	↑↑ 4	↑ 3	OceanBase +	关系型	HP			137	17	536.72	+49.07	+46.78
3	↓ 2	↓ 2	openGauss +	关系型	TP			562	65	535.70	-29.27	+1.08
4	↓ 3	4	达梦 +	关系型	TP			381	0	535.22	-17.05	+90.28
5	5	5	GaussDB +	关系型	HP			562	65	473.27	+27.55	+41.13
6	6	6	PolarDB +	关系型	HP			512	26	441.38	+22.60	+89.00
7	7	↑↑ 9	人大金仓 +	关系型	TP			232	0	418.47	+23.27	+125.79
8	8	↓ 7	GBase +	关系型	TP			152	0	285.87	-21.47	-50.16
9	9	↓ 8	TDSQL +	关系型	HP			39	10	277.26	+26.31	-24.81
10	10	10	AnalyticDB +	关系型	AP			480	28	206.16	+4.83	+8.09

来源：中国信息通信研究院，2021 年 6

图 8 中国数据库市场规模及增速

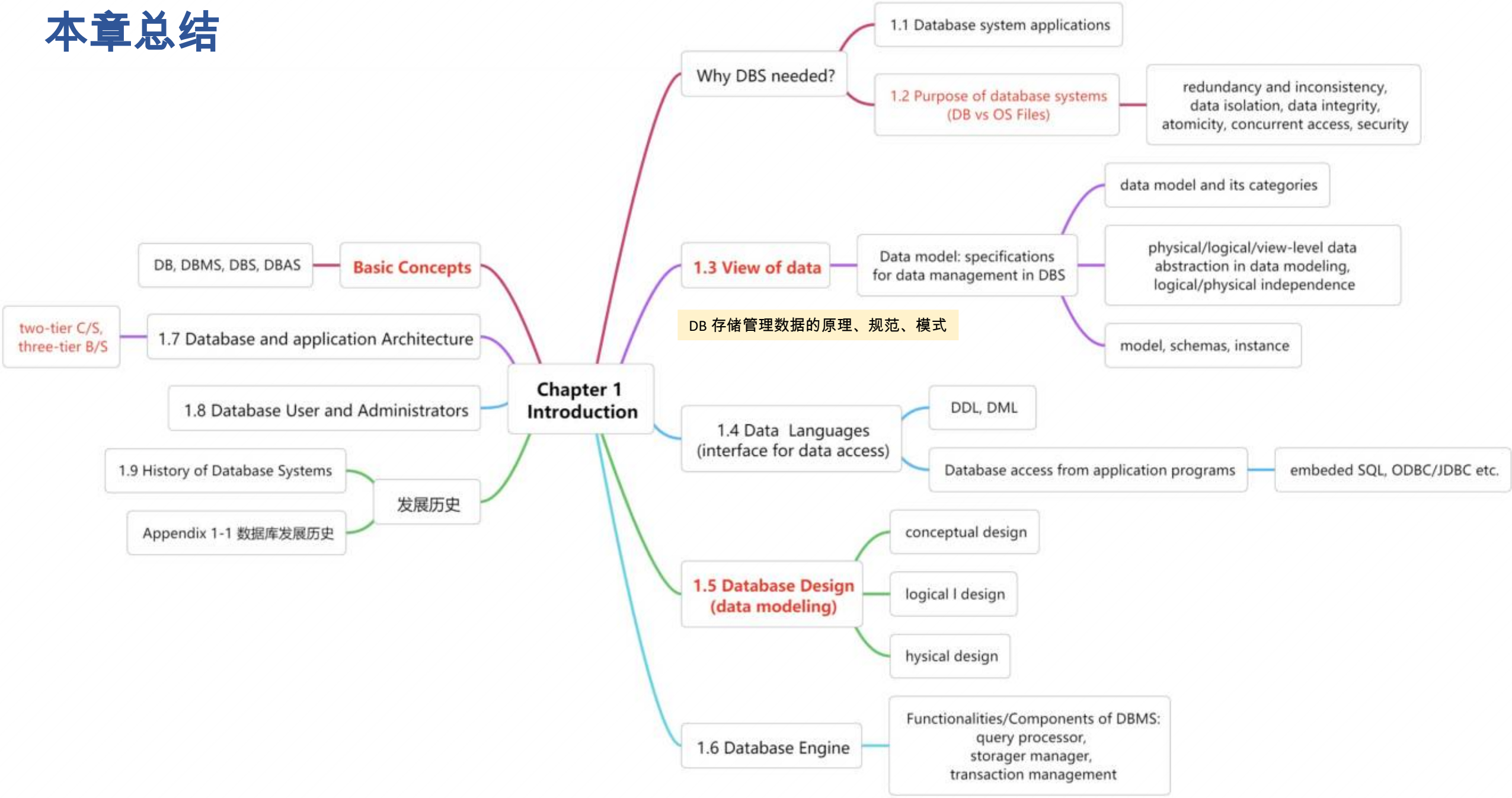


来源：中国信息通信研究院，2021 年 6 月

图 13 我国数据库产品分布情况

我国关系型数据库产品多数基于 MySQL 和 PostgreSQL 二次开发而来。据中国信通院统计分析，截止 2021 年 6 月，我国数据库产品共有 135 款。其中关系型数据库 81 个，非关系型数据库有 54 个，

本章总结


Chapter 1 Introduction
Why DBS needed?
1.1 Database system applications
1.2 Purpose of database systems (DB vs OS Files)
redundancy and inconsistency, data isolation, data integrity, atomicity, concurrent access, security
1.3 View of data
Data model: specifications for data management in DBS
data model and its categories
physical/logical/view-level data abstraction in data modeling, logical/physical independence
model, schemas, instance
DB 存储管理数据的原理、规范、模式
1.4 Data Languages (interface for data access)
DDL, DML
Database access from application programs
embeded SQL, ODBC/JDBC etc.
1.5 Database Design (data modeling)
conceptual design
logical l design
hysical design
1.6 Database Engine
Functionalities/Components of DBMS: query processor, storager manager, transaction management
Basic Concepts
DB, DBMS, DBS, DBAS
1.7 Database and application Architecture
two-tier C/S, three-tier B/S
1.8 Database User and Administrators
发展历史
1.9 History of Database Systems
Appendix 1-1 数据库发展历史

Thanks for your attention